

PagBus — Quadro Trello

Versão MVP para demonstração acadêmica

Sobre o projeto

Aplicativo mobile de pagamento de passagem de ônibus via saldo virtual.

O usuário se cadastra com CPF + senha, recebe um QR Code único (funciona como cartão), faz recargas de saldo e usa o QR Code para pagar a passagem na catraca simulada.

A catraca é simulada por uma tela no notebook que lê o QR Code pela câmera e valida o saldo.

Sem Arduino nesta versão — apenas software.

Legenda de integrantes

-  INT-1 — Dados, Infraestrutura e Banco de Dados
-  INT-2 — Backend e Integração com a Catraca
-  INT-3 — Frontend Flutter, UX e Qualidade

Rotação de Scrum Master

Sprint	Scrum Master
Sprint 1	● INT-1
Sprint 2	● INT-2
Sprint 3	● INT-3
Sprint 4	● INT-1
Sprint 5	● INT-2 ou ● INT-3

LISTA — BACKLOG / IDEIAS FUTURAS

(funcionalidades para depois que o MVP estiver funcionando)

Cartão: Integração com Arduino

Adicionar servo motor, LEDs e buzzer físico para simular catraca real.

● INT-2

Checklist:

- ☐ ● Instalar Arduino IDE
- ☐ ● Montar circuito: servo SG90, LED verde, LED vermelho, buzzer
- ☐ ● Código Arduino: aguarda caractere via Serial e aciona componentes
- ☐ ● Comunicar Python com Arduino via pyserial
- ☐ ● Documentar esquema de ligação dos componentes

Cartão: Pagamento Pix real

Substituir saldo virtual por integração com gateway de pagamento real.

● INT-2

Checklist:

- ☐  Pesquisar gateways: Mercado Pago, Stripe, Asaas
- ☐  Implementar geração de QR Code Pix real
- ☐  Webhook para confirmar pagamento aprovado
- ☐  Atualizar saldo após confirmação do banco

Cartão: Histórico de uso detalhado

Usuário vê data, hora e linha de cada passagem utilizada.

  INT-3 (tela) +  INT-2 (endpoint) +  INT-1 (tabela)

Checklist:

- ☐  Criar tabela historico_uso (usuario_id, data_hora, valor_debitado)
- ☐  GET /usuarios/me/historico
- ☐  Tela de histórico detalhado no app

Cartão: Perfis de cartão (estudante, idoso, PCD)

Diferentes tarifas por perfil com desconto automático ao debitar.

  INT-1 (banco) +  INT-2 (lógica) +  INT-3 (tela)

Checklist:

- ☐  Adicionar campo tipo_cartao na tabela usuarios
- ☐  Lógica de desconto por perfil no débito de passagem
- ☐  Tela de seleção de perfil no cadastro

LISTA — FASE 1: FUNDAMENTOS (Semanas 1-3)

Sprint 1 — todos estudam em paralelo

Cartão: APRENDER DART E FLUTTER BÁSICO

  INT-3

Checklist:

- ☐ ● Instalar Flutter SDK e configurar VS Code
- ☐ ● Completar curso introdutório (flutter.dev/learn)
- ☐ ● Entender widgets: StatelessWidget e StatefulWidget
- ☐ ● Criar demo: tela simples com cartão fake e botão de recarga
- ☐ ● Aprender navegação entre telas com go_router
- ☐ ● Entender gerenciamento de estado com Riverpod
- ☐ ● Criar tela de loading e feedback visual (animação de sucesso/erro)

Cartão: APRENDER JAVASCRIPT/TYPESCRIPT

👤 ● INT-2

Checklist:

- ☐ ● Variáveis, Funções, Arrays e Objetos
- ☐ ● Promises e Async/Await
- ☐ ● Módulos ES6 (Import/Export)
- ☐ ● Entender npm e package.json
- ☐ ● Instalar Node.js e rodar primeiro script

Cartão: CONFIGURAR AMBIENTE DE DESENVOLVIMENTO

👤 Todos (● INT-1 + ● INT-2 + ● INT-3)

Checklist:

- ☐ ● ● Instalar Node.js LTS
- ☐ ● Instalar Flutter SDK
- ☐ ● ● Instalar VS Code com extensões: Dart, Flutter, REST Client
- ☐ ● ● Instalar DBeaver (cliente de banco de dados)
- ☐ ● Criar repositório no GitHub (branches: main e develop)
- ☐ Todos aceitar convite do repositório

Cartão: ENTENDER API REST E JSON

  INT-2 (lidera) +  INT-3 (aprende para consumir no app)

Checklist:

- ☐  Estudar verbos HTTP: GET, POST, PUT, DELETE
 - ☐   Entender JSON e como os dados trafegam
 - ☐   Instalar Insomnia e testar API pública
 - ☐  Entender status codes: 200, 201, 400, 401, 403, 404 e 500
 - ☐  Escrever arquivo API.md no GitHub com contratos dos endpoints (usado pelo INT-3 para desenvolver telas em paralelo)
-

LISTA — FASE 2: BANCO DE DADOS (Semanas 2-3)

Pode rodar em paralelo com a Fase 1

Cartão: MODELAGEM E CRIAÇÃO DO BANCO

  INT-1 (lidera) +  INT-2 (valida schema)

Checklist:

- ☐  Instalar PostgreSQL localmente
 - ☐  Instalar DBeaver e conectar ao banco
 - ☐  Criar tabela usuarios (id, nome, cpf, email, senha_hash, saldo, qr_code_token, criado_em)
 - ☐  Criar tabela recargas (id, usuario_id, valor, metodo, status, criado_em)
 - ☐  Criar tabela validacoes (id, usuario_id, valor_debitado, resultado, validado_em)
 - ☐  Inserir dados de demonstração: 2-3 usuários com saldo pré-populado
 - ☐  Validar schema com os endpoints planejados
 - ☐  Documentar diagrama ER no README do GitHub
-

LISTA — FASE 3: BACKEND (Semanas 3-6)

Cartão: CRIAR SERVIDOR NODE.JS + EXPRESS

  INT-2

Checklist:

- ☐  npm init e instalar Express + TypeScript
- ☐  Criar estrutura: /routes, /controllers, /models, /middleware
- ☐  Criar rota GET /health que retorna { status: "ok", projeto: "PagBus" }
- ☐  Configurar variáveis de ambiente com dotenv
- ☐  Configurar tsx watch para reinício automático

Cartão: AUTENTICAÇÃO — CPF E SENHA

  INT-2 (endpoints) +  INT-1 (banco)

Checklist:

- ☐  Instalar jsonwebtoken e bcrypt
- ☐  POST /auth/registro — recebe nome, CPF, email, senha
- ☐  Validar formato de CPF antes de salvar
- ☐  Salvar senha com bcrypt (nunca salvar senha pura)
- ☐  Gerar QR Code token único para o usuário (UUID) no momento do cadastro
- ☐  Salvar usuário no banco com saldo inicial de R\$0,00
- ☐  POST /auth/login — recebe CPF + senha, retorna JWT
- ☐  Criar middleware autenticar — valida JWT e injeta usuário na requisição
- ☐  Testar registro e login no Insomnia

Cartão: ENDPOINTS DO USUÁRIO E SALDO

  INT-2 (endpoints) +  INT-1 (queries no banco)

Checklist:

- ☐  GET /usuarios/me — retorna nome, CPF, saldo e qr_code_token
- ☐  PUT /usuarios/me — atualiza nome e email
- ☐  DELETE /usuarios/me — exclui conta e todos os dados
- ☐  Testar todos no Insomnia

Cartão: ENDPOINTS DE RECARGA

  INT-2 (lógica) +  INT-1 (banco)

Checklist:

- ☐  POST /recargas — recebe valor e método (simulado), adiciona saldo ao usuário
- ☐  Valores aceitos: R\$5, R\$10, R\$20, R\$50 — rejeitar outros valores
- ☐  Simular aprovação instantânea (sem gateway real nesta versão)
- ☐  Registrar recarga na tabela recargas com status "aprovado"
- ☐  Atualizar saldo do usuário no banco
- ☐  GET /recargas — lista recargas do usuário logado
- ☐  Testar todos no Insomnia

Cartão: ENDPOINT DE VALIDAÇÃO DO QR CODE (CATRACA)

  INT-2 — endpoint mais importante do sistema

Checklist:

- ☐  POST /validar — recebe qr_code_token lido pela câmera
- ☐  Buscar usuário pelo qr_code_token no banco
- ☐  Se usuário não encontrado: retornar { resultado: "NEGAR", motivo: "QR Code inválido" }
- ☐  Se saldo < valor da passagem (R\$4,50): retornar { resultado: "NEGAR", motivo: "Saldo insuficiente" }
- ☐  Se saldo suficiente: debitar R\$4,50, retornar { resultado: "LIBERAR", saldo_restante: X }
- ☐  Registrar tentativa na tabela validacoes (válido ou negado)
- ☐  Testar cenários: QR Code válido, inválido, saldo insuficiente no Insomnia

LISTA — FASE 4: APLICATIVO FLUTTER (Semanas 4-8)

Cartão: ESTRUTURA DE NAVEGAÇÃO

  INT-3

Checklist:

- ☐  Instalar go_router para navegação
- ☐  Definir telas: Onboarding, Login, Cadastro, Home, Recarga, Cartão/QR Code
- ☐  Criar estado global de autenticação com Riverpod
- ☐  Persistir sessão com FlutterSecureStorage (usuário não loga toda vez)
- ☐  Rotas protegidas: redirecionar para login se não autenticado

Cartão: TELA DE ONBOARDING

  INT-3

Checklist:

- ☐  2-3 slides: "Cadastre-se", "Recarregue", "Pague com QR Code"
- ☐  Botão "Criar conta" e botão "Já tenho conta"
- ☐  Salvar flag onboarding_visto para não exibir novamente

Cartão: TELA DE CADASTRO

  INT-3 (tela) +  INT-2 (endpoint POST /auth/registro)

Checklist:

- ☐  Campos: nome completo, CPF, email, senha, confirmar senha
- ☐  Máscara de CPF no campo (formato 000.000.000-00)
- ☐  Validação em tempo real: CPF válido, senhas coincidem, mínimo 6 caracteres
- ☐  Ao cadastrar com sucesso: navegar para tela Home
- ☐  Exibir erro amigável (CPF já cadastrado, email inválido, etc.)

Cartão: TELA DE LOGIN

  INT-3 (tela) +  INT-2 (endpoint POST /auth/login)

Checklist:

- ☐  Campos: CPF e senha
- ☐  Máscara de CPF
- ☐  Salvar token JWT no FlutterSecureStorage após login
- ☐  Link "Criar conta"
- ☐  Exibir mensagem de erro amigável (CPF não encontrado, senha incorreta)

Cartão: TELA HOME (SALDO E AÇÕES)

  INT-3 (tela) +  INT-2 (GET /usuarios/me)

Checklist:

- ☐  Exibir nome do usuário e saldo atual
- ☐  Botão principal "Recarregar" — abre tela de recarga
- ☐  Botão "Meu Cartão" — abre tela do cartão/QR Code
- ☐  Indicador visual de saldo baixo (saldo menor que R\$4,50)
- ☐  Buscar dados do usuário na API ao abrir a tela
- ☐  Pull-to-refresh para atualizar saldo

Cartão: TELA DO CARTÃO (FLIP CARD + QR CODE)

  INT-3 — funcionalidade visual central do app

Checklist:

- ☐  Exibir cartão estilizado com nome do usuário e saldo (frente do cartão)
- ☐  Ao tocar no cartão: animação de flip 3D mostrando o verso
- ☐  Verso do cartão: QR Code gerado a partir do qr_code_token do usuário
- ☐  QR Code grande e legível para ser lido pela câmera
- ☐  Botão para voltar para a frente do cartão (ou toque novamente)
- ☐  Manter tela acesa enquanto QR Code estiver visível (wakelock)
- ☐  Exibir aviso de saldo insuficiente se saldo < R\$4,50

Cartão: TELA DE RECARGA

  INT-3 (tela) +  INT-2 (POST /recargas) +  INT-1 (atualiza banco)

Checklist:

- ☐  Exibir opções de valor: R\$5 / R\$10 / R\$20 / R\$50
 - ☐  Destaque visual no valor selecionado
 - ☐  Botão "Confirmar Recarga"
 - ☐  Loading enquanto aguarda resposta do backend
 - ☐  Ao confirmar: exibir animação de sucesso e novo saldo
 - ☐  Ao erro: exibir mensagem amigável
 - ☐  Após sucesso: voltar para Home com saldo atualizado
-

LISTA — FASE 5: CATRACA SIMULADA NO NOTEBOOK (Semanas 6-9)

Cartão: LEITOR DE QR CODE PELA CÂMERA

  INT-2

Checklist:

- ☐  Escolher tecnologia: Python com OpenCV + pyzbar ou Node.js com jimp
- ☐  Instalar dependências: pip install opencv-python pyzbar (se Python)
- ☐  Abrir câmera do notebook em loop contínuo
- ☐  Detectar e decodificar QR Code em tempo real
- ☐  Extrair o token do QR Code decodificado
- ☐  Enviar POST /validar ao backend com o token lido
- ☐  Receber resposta: LIBERAR ou NEGAR
- ☐  Testar leitura com QR Code impresso e pelo celular

Cartão: INTERFACE DA CATRACA (TELA DO NOTEBOOK)

  INT-2 (lógica) +  INT-3 (suporte no visual se necessário)

A tela que fica aberta no notebook durante a demonstração.

Checklist:

- ☐  Criar interface em HTML + JavaScript ou Python com tkinter/pygame
- ☐  Estado inicial: tela preta com texto "Aproxime o QR Code" e ícone de câmera
- ☐  Ao ler QR Code válido (LIBERAR):
- ☐  Tela fica verde com texto "ACESSO LIBERADO" e ícone de check
- ☐  Emitir 1 BIP curto (beep via sistema)
- ☐  Exibir nome do usuário e saldo restante
- ☐  Após 3 segundos: voltar para tela preta
- ☐  Ao ler QR Code inválido ou saldo insuficiente (NEGAR):
- ☐  Tela fica vermelha com texto "ACESSO NEGADO" e ícone de X
- ☐  Emitir 3 BIPS (beep via sistema)
- ☐  Exibir motivo (saldo insuficiente / QR Code inválido)
- ☐  Após 3 segundos: voltar para tela preta

Cartão: BIP DO SISTEMA (SEM ARDUINO)

  INT-2

Checklist:

- ☐  Python: usar winsound.Beep() no Windows ou subprocess para beep no Linux
 - ☐  Ou: reproduzir arquivo de som .wav curto com pygame
 - ☐  Definir tom diferente para LIBERAR (1 bip grave) e NEGAR (3 bips agudos)
 - ☐  Testar em todos os notebooks da equipe
-

LISTA — FASE 6: INTEGRAÇÃO E TESTES (Semanas 8-10)

Cartão: TESTE DO FLUXO COMPLETO

 Todos ( INT-1 +  INT-2 +  INT-3)

Checklist:

- ☐  App: usuário abre o app e faz cadastro com CPF
- ☐  App: usuário realiza recarga de R\$10
- ☐  App: usuário abre tela do cartão e toca para ver QR Code
- ☐  Notebook: câmera detecta QR Code e envia token ao backend
- ☐  Backend: valida saldo, debita R\$4,50, retorna LIBERAR
- ☐  Notebook: tela fica verde, 1 BIP, exibe nome e saldo restante
- ☐  Testar com saldo insuficiente (deve negar com 3 BIPS e tela vermelha)
- ☐  Testar com QR Code inválido (foto de QR Code aleatório)
- ☐  Testar recarga e uso em sequência várias vezes
- ☐  Registrar tempo do fluxo completo (meta: menos de 3 segundos)

Cartão: PREPARAÇÃO DA DEMONSTRAÇÃO

  INT-1 (dados e roteiro) +  INT-2 (hardware/notebook) +  INT-3 (app e slides)

Checklist:

- ☐  Preparar roteiro de demonstração de 5-10 minutos
- ☐  Popular banco com 2-3 contas demo com saldos variados
- ☐  Configurar notebook com câmera posicionada e interface da catraca aberta
- ☐  Testar tudo em rede local (celular e notebook na mesma rede Wi-Fi)
- ☐  Testar app em celular físico (não emulador)
- ☐  Preparar slides explicando a arquitetura do sistema
- ☐  Diagrama do fluxo: App → Backend → Câmera → Resposta visual + BIP

Cartão: DOCUMENTAÇÃO TÉCNICA

  INT-1 (lidera) + todos participam

Checklist:

- ☐  README.md com visão geral e instruções de instalação
 - ☐  Diagrama de arquitetura (App + Backend + Câmera + Notebook)
 - ☐  Documentar endpoints da API (arquivo API.md)
 - ☐  Documentar esquema do banco de dados (diagrama ER)
 - ☐  Criar vídeo demonstração de até 3 minutos como backup para a apresentação
-

LISTA — EM ANDAMENTO

LISTA — CONCLUÍDO

Resumo de responsabilidades por integrante

INT-1 — Dados, Infraestrutura e Banco de Dados

Fases com liderança: Fase 2 completa (Banco) + Documentação + suporte em testes.

Tarefas principais: modelagem e criação das tabelas (usuarios, recargas, validacoes), setup do PostgreSQL, inserção de dados de demonstração, diagrama ER, documentação da arquitetura, escrita do API.md no GitHub, testes de cenários negativos (saldo insuficiente, QR Code inválido) e registro de tempo de fluxo.

INT-2 — Backend e Integração com a Catraca

Fases com liderança: Fase 3 completa (Backend) + Fase 5 completa (Catraca Simulada).

Tarefas principais: servidor Node.js/Express, autenticação CPF+senha com JWT, endpoints de recarga e validação de QR Code, leitura de QR Code pela câmera com Python/OpenCV, interface da catraca no notebook (tela verde/vermelha), sistema de BIPs, lógica de débito de saldo.

INT-3 — Frontend Flutter, UX e Qualidade

Fases com liderança: Fase 4 completa (App Flutter).

Tarefas principais: todas as telas (onboarding, login, cadastro, home, cartão com flip card + QR Code, recarga), animação de flip 3D do cartão, wakelock na tela do QR Code, feedback visual de sucesso e erro, build do app para demonstração, slides de apresentação.